

Schéma de base de données

Overdrive

Aventix
Epitech Paris
9 février 2026

Sommaire

OverDrive – Schéma de base de données

0-Objectifs

1- Architecture globale : persistance vs temps réel

- 1- Données persistantes (PostgreSQL)
- 2- Données live (hors SQL)
- 3- Schéma : Architecture globale (PostgreSQL vs Redis vs Time-series)

2- Schéma PostgreSQL : tables principales

- 1- **users**
- 2- **auth_sessions** (ou **refresh_tokens**)
- 3- **providers**
- 4- **provider_connections**
- 5- **events**
- 6- **sessions**
- 7- **drivers**
- 8- **user_presets**
- 9- **preset_widgets**
- 10- **race_highlights**
- 11- Schéma : Architecture globale (PostgreSQL vs Redis vs Time-series)

3- Stockage temps réel (hors PostgreSQL)

- 1- Redis (recommandé)
- 2- Base time-series (optionnelle)
- 3- Schéma : Structure temps réel Redis (clés)

4- Conclusion

Objectif

L'objectif de la base de données d'OverDrive est de stocker l'ensemble des informations **persistantes** nécessaires au fonctionnement du produit, tout en conservant une architecture **scalable** et compatible avec des usages **temps réel**.

La base de données doit couvrir les besoins suivants :

- Gestion des utilisateurs et des sessions d'authentification
- Gestion des presets et layouts personnalisés
- Gestion des providers (championnats) et de leurs configurations
- Gestion des courses et sessions (métadonnées, calendrier, historique)
- (Optionnel) stockage d'événements marquants, highlights, et replays

OverDrive s'appuie sur une logique importante : les données "live" à haute fréquence (télémétrie, état temps réel, positions actualisées très souvent) ne sont pas adaptées à un stockage SQL classique à chaque mise à jour. Elles sont donc traitées via un système séparé, principalement :

- **Redis** pour le cache et l'état live,
- une base **time-series** optionnelle pour l'historique (InfluxDB ou Timescale).

Cette séparation garantit un excellent compromis entre performance, coût, et évolutivité.

1- Architecture globale : persistance vs temps réel

Le schéma de base de données d'OverDrive repose sur une séparation claire entre :

1- Données persistantes (PostgreSQL)

PostgreSQL est utilisé pour stocker toutes les données structurées et durables, notamment :

- comptes utilisateurs,
- profils,
- presets,
- catalogue de courses,
- configuration providers,
- événements marquants.

Ces données sont essentielles au produit, car elles permettent :

- la continuité de l'expérience,
- la personnalisation,
- la gestion multi-championnats,
- l'historique et la navigation dans le calendrier.

2- Données live (hors SQL)

Les données live sont caractérisées par :

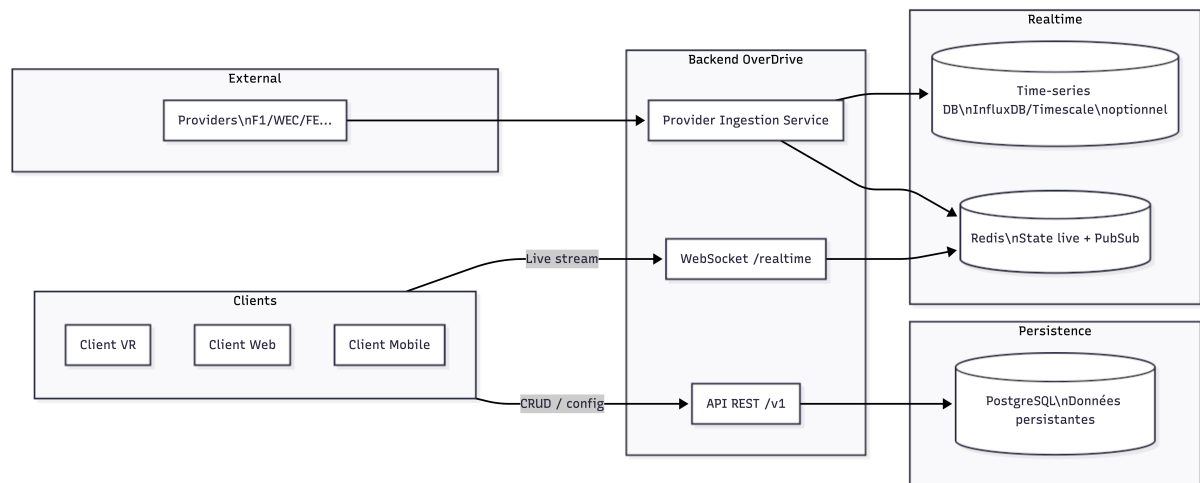
- une fréquence de mise à jour très élevée,
- des besoins de latence extrêmement faibles,
- un volume important.

Stocker ces données directement dans PostgreSQL à chaque mise à jour créerait :

- des coûts importants,
- des problèmes de performance,
- une complexité de maintenance inutile.

Pour cette raison, l'état live est stocké dans **Redis**, et l'historique télémétrie est éventuellement stocké dans une base time-series.

3- Shéma : Architecture globale (PostgreSQL vs Redis vs Time-series)



2- Tables principales (PostgreSQL)

Le schéma SQL est conçu pour être **modulaire** et **évolutif**, afin d'ajouter facilement de nouveaux providers, de nouvelles fonctionnalités, ou de nouveaux widgets.

1- Table users

La table **users** constitue le cœur de la gestion utilisateur.

Elle stocke :

- l'identifiant unique de l'utilisateur,
- les informations de connexion,
- les informations de profil,

- les timestamps de création et modification.

Champs principaux :

- **id** (UUID, PK)
- **email** (unique)
- **display_name**
- **password_hash** (si authentification classique, optionnel si OAuth)
- **created_at, updated_at**

2- Table auth_sessions (ou refresh_tokens)

OverDrive nécessite une gestion sécurisée des sessions.

Pour cela, une table dédiée permet de gérer la persistance de connexion via refresh tokens ou sessions longues.

Champs principaux :

- **id** (UUID, PK)
- **user_id** (FK → users.id)
- **refresh_token_hash**
- **expires_at**
- **created_at**

Cette séparation entre les données utilisateur et les données de session permet :

- de sécuriser la gestion des tokens,
- de limiter les impacts en cas de fuite,
- de gérer facilement les expirations,
- de révoquer des sessions sans modifier le profil utilisateur.

3- Table providers

OverDrive étant conçu comme une plateforme multi-championnats, la table **providers** référence les sources de données disponibles.

Chaque provider correspond à un championnat ou un système de données : F1, WEC, FE, WRC, etc.

Champs principaux :

- **id** (UUID, PK)
- **code** (ex: f1, wec, fe) unique
- **name**
- **status** (active, beta, disabled)
- **created_at**

Ce modèle permet d'ajouter un nouveau championnat sans modifier la structure globale de la base.

4- Table provider_connections

La table **provider_connections** représente la configuration d'un provider pour un utilisateur.

Elle permet de stocker des préférences par utilisateur, par exemple :

- paramètres régionaux,
- options d'affichage spécifiques,
- préférences de flux,
- configuration de récupération des données.

Champs principaux :

- **id** (UUID, PK)
- **user_id** (FK)
- **provider_id** (FK)
- **config_json** (JSONB)
- **created_at, updated_at**

Un point important est la gestion des données sensibles.

Dans la mesure du possible, OverDrive évite de stocker des tokens externes en base.

Si cela devient nécessaire, ils doivent être chiffrés, et stockés séparément dans un système sécurisé.

5- Table events

La table **events** correspond au catalogue des courses ou événements (ex : Grand Prix, manche WEC, rallye, etc.).

Elle permet d'afficher :

- un calendrier,
- des informations de contexte,
- l'historique,
- la course en cours ou à venir.

Champs principaux :

- **id** (UUID, PK)
- **provider_id** (FK)

- **season_year**
- **name** (ex: "Bahrain GP")
- **location**
- **start_time_utc**
- **end_time_utc** (nullable)
- **status** (scheduled, live, finished)
- **created_at**

6- Table sessions

Un **event** peut être composé de plusieurs sessions :

- Practice
- Qualifying
- Race

Cette séparation est essentielle, car le live timing et les données temps réel doivent être liés à une session précise.

Champs principaux :

- **id** (UUID, PK)
- **event_id** (FK)
- **type** (race, quali, practice)
- **status** (live, finished)
- **started_at_utc, ended_at_utc**

7- Table drivers

La table **drivers** constitue un référentiel des pilotes, par provider.

Elle est nécessaire car :

- un même pilote peut avoir un ID différent selon le provider,
- l'application doit afficher des informations constantes,
- il est important de ne pas dépendre uniquement des flux live pour l'affichage.

Champs principaux :

- **id** (UUID, PK)
- **provider_id** (FK)

- **display_name**
- **team_name**
- **number** (nullable)

Une extension possible du schéma serait d'ajouter une table **teams**, afin de structurer plus proprement les écuries, mais ce n'est pas indispensable en V1.

8- Table user_presets

La table **user_presets** permet de stocker les presets d'interface de chaque utilisateur. Un preset correspond à une configuration de cockpit informationnel : choix des widgets, disposition, organisation, etc.

Champs principaux :

- **id** (UUID, PK)
- **user_id** (FK)
- **name**
- **is_default** (bool)
- **created_at, updated_at**

Cette table permet à l'utilisateur :

- de créer plusieurs configurations,
- de choisir un preset par défaut,
- de conserver ses réglages entre sessions.

9- Table preset_widgets

La table **preset_widgets** représente la composition d'un preset. Chaque ligne correspond à un widget affiché dans l'interface, avec :

- son type,
- sa configuration,
- sa position et transformation en VR,
- son ordre d'affichage.

Champs principaux :

- **id** (UUID, PK)
- **preset_id** (FK)
- **widget_type** (ex : standings, telemetry, track, highlights)
- **settings_json** (JSONB)
- **pos_x, pos_y, pos_z** (float)

- **rot_x, rot_y, rot_z** (float)
- **scale** (float)
- **order_index** (int)

L'usage combiné de champs dédiés (transform VR) et de JSONB (paramètres spécifiques) permet une grande flexibilité :

- ajout de nouveaux widgets sans changer la base,
- stockage de paramètres variables selon le type de widget,
- adaptation rapide aux évolutions de l'interface.

10- Table **race_highlights**

La table **race_highlights** permet de stocker les événements marquant d'une session:

- dépassements,
- pit stops,
- crash,
- drapeaux,
- incidents.

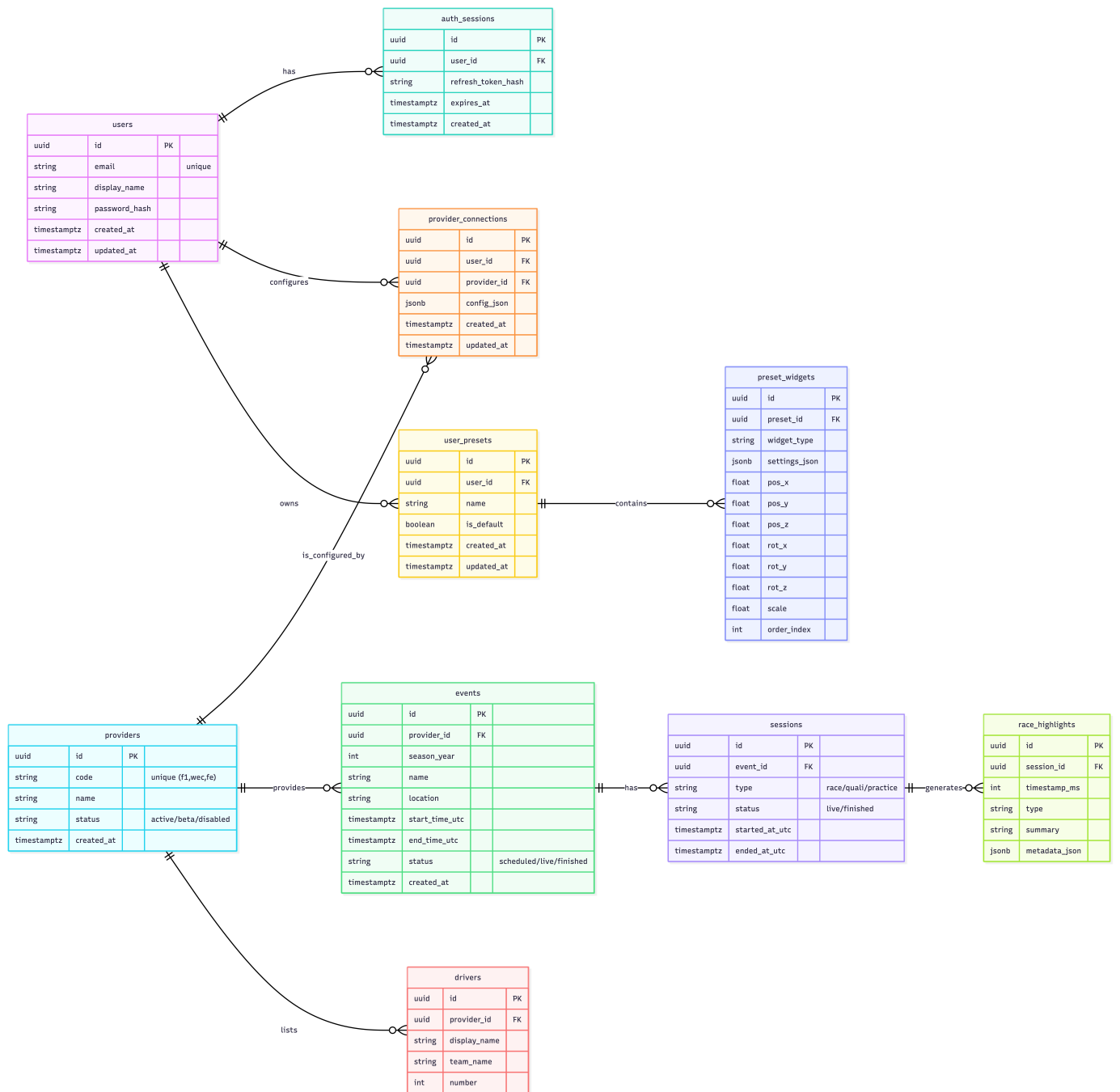
Champs principaux :

- **id** (UUID, PK)
- **session_id** (FK)
- **timestamp_ms**
- **type** (overtake, pit, crash, flag)
- **summary**
- **metadata_json** (JSONB)

Cette table est particulièrement utile pour :

- construire un "résumé" de course,
- permettre une navigation rapide dans les moments clés,
- alimenter une fonctionnalité de replay ou d'analyse.

10- Schéma : ERD PostgreSQL (tables + relations)



3. Stockage temps réel (hors PostgreSQL)

Comme mentionné précédemment, OverDrive distingue le stockage persistant du stockage live.

1- Redis (recommandé)

Redis est utilisé pour stocker et distribuer rapidement l'état live.

Exemples de clés :

- **realtime:session:<sessionId>:state**
(classement, drapeaux, tours, statut global)
- **realtime:session:<sessionId>:driver:<driverId>**
(dernier état pilote : position, temps, pneus, etc.)

Redis permet :

- une latence extrêmement faible,
- un accès rapide pour plusieurs clients simultanés,
- une gestion efficace de l'état temps réel.

2- Base time-series (optionnelle)

Si OverDrive propose une fonctionnalité d'analyse avancée ou de "replay data", une base orientée séries temporelles peut être ajoutée.

Solutions possibles :

- InfluxDB
- Timescale (extension PostgreSQL)

Ce stockage permet de conserver :

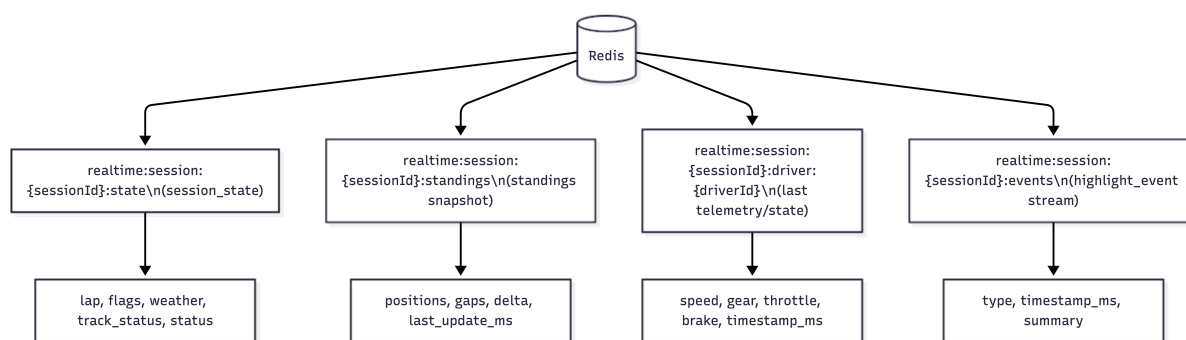
- la télémétrie historique,
- les courbes de performance,
- les comparaisons de tours,
- les analyses post-course.

Cette partie est optionnelle car elle implique :

- plus de coûts,
- plus de complexité,
- plus de volume de données.

Cependant, elle constitue une extension naturelle pour un produit premium orienté data.

3- Schéma : Structure temps réel Redis (clés)



Conclusion

Le schéma de base de données d'OverDrive est conçu pour être :

- **modulaire** : ajout facile de nouveaux providers et fonctionnalités,
- **robuste** : séparation claire entre persistance et live,
- **scalable** : stockage spécialisé selon les usages,
- **orienté produit** : support complet de la personnalisation via presets.

PostgreSQL couvre toutes les données essentielles (utilisateurs, providers, événements, presets), tandis que Redis et une base time-series optionnelle permettent de répondre aux contraintes temps réel et aux besoins futurs d'analyse.

Cette architecture de données constitue une base solide pour supporter l'évolution d'OverDrive, son ambition premium, et son extension à plusieurs championnats.

